

Основы анализа эффективности алгоритмов (Введение в теорию сложности алгоритмов)

*Made by Khk_09
Special thanks to Come Off*

Временная эффективность

- Сколько времени работает алгоритм
- Скорее всего будем считать в операциях суммирования и `swap`
- Не считаем в секундах, так как зависит от многих внешних факторов (загруженность системы, кэш-память)

Пространственная эффективность

- Сколько оперативной памяти требует

Оценка размера входных данных

- Сколько байтов поступило на вход
- В алгоритмах можно считать за количество элементов

Когда говорят об эффективности

- В худшем случае (C_{worst})
- В лучшем случае
- Средний случай

O -обозначение

Определение. Говорят, что функция $t(n)$ принадлежит множеству $O(g(n))$, что записывается как $t(n) \in O(g(n))$, если для всех больших значений n функция $t(n)$ ограничена сверху некоторой константой, умноженной на значение функции $g(n)$, т.е. существует положительная константа c и неотрицательное целое число n_0 такое, что $t(n) \leq cg(n)$ для всех $n \geq n_0$.

Пример

$$t(n) = n^2 + 3n + 1$$

$$g(n) = n^2$$

$$n^2 + 3n + 1 \leq 2n^2, c = 2$$

$$0 \leq n^2 - 3n - 1$$

Ω -обозначение

Определение. Говорят, что функция $t(n)$ принадлежит множеству $\Omega(g(n))$, что записывается как $t(n) \in \Omega(g(n))$, если для всех больших значений n функция $t(n)$ ограничена снизу некоторой константой, умноженной на значение функции $g(n)$, т.е. существует положительная константа c и неотрицательное целое число n_0 такое, что $t(n) \geq cg(n)$ для всех $n \geq n_0$.

Θ -обозначение

Определение. Говорят, что функция $t(n)$ принадлежит множеству $\Theta(g(n))$, что записывается как $t(n) \in \Theta(g(n))$, если для всех больших значений n функция $t(n)$ ограничена снизу и сверху некоторыми константами, умноженными на значение функции $g(n)$, т.е. существуют положительные константы c_1 и c_2 и неотрицательное целое число n_0 такое, что $c_1g(n) \leq t(n) \leq c_2g(n)$ для всех $n \geq n_0$.

Оценка через пределы

$$t(n) \in O(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{g(n)}{t(n)} > 0$$

$$t(n) \leq cg(n) \\ \frac{1}{c} \leq \frac{g(n)}{t(n)}$$

Общая оценка

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \begin{cases} 0 & \text{если } t(n) \text{ имеет меньший порядок роста, чем } g(n) \Rightarrow t(n) \in O(g(n)) \\ c & \text{если } t(n) \text{ имеет тот же порядок роста, что и } g(n) \Rightarrow t(n) \in O(g(n)) \\ & \text{и } t(n) \in \Omega(g(n)) \Rightarrow t(n) \in \Theta(g(n)) \\ \infty & \text{если } t(n) \text{ больший порядок роста, чем } g(n) \Rightarrow t(n) \in \Omega(g(n)) \end{cases}$$

Правило Лопиталя

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{t'(n)}{g'(n)}$$

Формула Стирлинга

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \text{ для больших чисел}$$

Теорема 1. Если $t_1(n) \in O(g_1(n))$ и $t_2(n) \in O(g_2(n))$, то $(t_1(n) + t_2(n)) \in O(\max\{g_1(n), g_2(n)\})$. Аналогичные утверждения справедливы также для множеств Ω и Θ .

Примеры:

$$1. t(n) = (n^2 + 1)^{10} \\ t(n) \in \Theta(n^{20})$$

$$2. t(n) = \sqrt{10n^2 + 7n + 3}$$

$$t(n) \in \Theta(n)$$

Замечание: почему не нужно указывать основание логарифма в асимптотике

Переход к другому основанию логарифма: $\log_a b = \frac{\log_c b}{\log_c a}$

Заметим, что $\frac{1}{\log_c a} = \text{const}$

Тогда $\log_a b = \text{const} * \log_c b \Rightarrow$ одинаковый порядок роста для логарифмов с разными основаниями

Немного про запись логарифмов

- $\log_a$ - обобщённая запись: основание (a) не указывается только если оно не важно
- lg - десятичный логарифм (по основанию 10)
- $\ln$ - натуральный логарифм (по основанию $e = 2.71828$)

Разбор задачи про дверь и бесконечную стену (2.2.10)

Давайте ходить в обе стороны на одинаковое количество шагов, увеличивая это количество в 2 раза после прохода этим расстоянием в обе стороны и возвращения в обратную сторону. Начальное количество: 1 Тогда мы получили $O(n)$. Докажем это
Рассмотрим худший случай: дверь находится в конце очередного прохода
Тогда количество итераций, за которое достигли дверь:

$$4 \lceil \log_2 n \rceil$$

4 - так как в худшем случае в начале ушли в другую сторону

Сумма шагов:

$$4 \frac{2^x - 1}{2 - 1}, x = \lceil \log_2 n \rceil \Rightarrow 4n - 4 \Leftrightarrow O(n)$$

$$\sum_{i=0}^{n-1} (i(i+1)) = \sum_{i=0}^{n-1} (i^2 + i) = \sum_{i=0}^{n-1} i^2 + \sum_{i=0}^{n-1} i = \frac{n(n-1)(2n-1)}{6} + \frac{n(n-1)}{2}$$

$$\sum_i^n \sum_j^n ij = \sum_i^n i \sum_j^n j = \sum_i^n i \frac{(n-j+1)(j+n)}{2} = \frac{n^2 - j^2 + n + j}{2} \frac{n^2 - i^2 + n - i}{2}$$

Основные формулы суммирования

$$\sum_{i=l}^n 1 = n - l + 1$$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4}$$

$$\sum_{i=1}^n i^k \approx \frac{1}{k+1} n^{k+1}$$

$$\sum_{i=0}^n a^i = \frac{a^{n+1} - 1}{a - 1}, (a \neq 1)$$

$$\sum_{i=1}^n i 2^i = (n-1)2^{n+1} + 2$$

$$\sum_{i=1}^n \frac{1}{i} \approx \ln n + \gamma, \gamma - \text{постоянная Эйлера и } \gamma = 0.577216$$

$$\sum_{i=1}^n \lg i \approx n \lg n$$

Правила работы с суммами

$$\sum_{i=l}^n c a_i = c \sum_{i=l}^n a_i$$

$$\sum_{i=l}^n (a_i \pm b_i) = \sum_{i=l}^n a_i \pm \sum_{i=l}^n b_i$$

$$\sum_{i=l}^n (c a_i + b_i) = c \sum_{i=l}^n a_i + \sum_{i=l}^n b_i$$

$$\sum_{i=l}^n a_i = \sum_{i=l}^m a_i + \sum_{i=m+1}^n a_i, l \leq m < n$$

$$\sum_{i=l}^n (a_i - a_{i-1}) = a_n - a_{l-1}$$

Немного практики и выводов

№ 2.3.1

$$\begin{aligned} \text{д) } \sum_{i=0}^{n-1} [i(i+1)] &= \sum_{i=0}^{n-1} [i^2 + i] = \frac{n(n-1)(2n-1)}{6} + \frac{n(n-1)}{2} = \frac{n(n-1)(2n+2)}{6} \\ &= \frac{n(n+1)(n-1)}{3} \end{aligned}$$

Порядока роста: $\sum_{i=1}^n \lg i$

$$\sum_{i=\frac{n}{2}}^n \lg i \leq i \sum_{i=1}^n \lg i \leq \sum_{i=1}^n \lg n = n \lg n$$

$$\sum_{i=\frac{n}{2}}^n \lg i \geq \sum_{i=\frac{n}{2}}^n \lg \frac{n}{2} = \frac{n}{2} \lg \frac{n}{2} = \frac{n}{2} (\lg n - \lg 2) = \frac{n \lg n}{2} - \frac{n \lg 2}{2}$$

$$\Rightarrow \frac{n \lg n}{2} - \frac{n \lg 2}{2} \leq \sum_{i=1}^n \lg i \leq n \lg n$$

$$\Rightarrow \sum_{i=1}^n \lg i \in \Theta(n \lg n)$$

Порядок роста: $\sum_{i=1}^n i^k$

$$\sum_{i=1}^n i^k \leq \sum_{i=1}^n n^k = n^{k+1}$$

$$\sum_{i=\frac{n}{2}}^n i^k \leq \sum_{i=1}^n i^k$$

Вывод: $\sum_{i=1}^n i 2^i$

$$S_n = \sum_{i=1}^n i 2^i$$

$$2S_n = \sum_{i=1}^n i 2^{i+1}$$

$$\begin{aligned}
2S_n - S_n &= \sum_{i=2}^{n+1} (i-1)2^i - \sum_{i=1}^n i2^i = n2^{n+1} + \sum_{i=2}^n (i-1)2^i - \sum_{i=2}^n i2^i - 2 = \\
&= n2^n - 2 + \sum_{i=2}^n [(i-1)2^i - i2^i] = n2^n - 2 \sum_{i=2}^n 2_i = n2^n - 2 - \frac{4(2^{n-1} - 1)}{1} = \\
&= n2^{n+1} - 2 - 2^{n+1} + 2 \in \Theta(n2^n)
\end{aligned}$$

№ 2.3.2

$$\begin{aligned}
c) \sum_{i=0}^{n-1} \sum_{j=0}^{i-1} [i+j] &= \sum_{i=0}^{n-1} \sum_{j=0}^{i-1} i + \sum_{i=0}^{n-1} \sum_{j=0}^{i-1} j = \sum_{i=0}^{n-1} i^2 + \sum_{i=0}^{n-1} \frac{i(i-1)}{2} = \\
&= \frac{n(n-1)(2n-1)}{6} + \frac{n(n-1)(2n-1)}{12} - \frac{n(n-1)}{4} = \\
&= \frac{n(n-1)(6n-6)}{12} = \frac{n(n-1)^2}{2} \in \Theta(n^3)
\end{aligned}$$

$$\begin{aligned}
f_1(n) &\in O(g_1(n)) \\
f_2(n) &\in O(g_2(n)) \\
\Rightarrow f_1(n)f_2(n) &\in O(g_1(n)g_2(n))
\end{aligned}$$

$$\begin{aligned}
\blacktriangle \\
C_1, N_1 : f_1(n) &\leq C_1 g_1(n) \\
C_2, N_2 : f_2(n) &\leq C_2 g_2(n)
\end{aligned}$$

$$\begin{cases} C = C_1 C_2 \\ N = \max(N_1, N_2) : f_1(n)f_2(n) \leq C g_1(n)g_2(n) = C_1 g_1(n) C_2 g_2(n) \end{cases}$$

■

$$3n^2 + 5n + 7 \in \Theta(n^2)$$

▲

$$\begin{cases} C_1 = 3 \\ C_2 = 15 : 3n^2 \leq 3n^2 + 5n + 7 \leq 3n^2 + 5n^2 + 7n^2 = 15n^2 \\ N = 1 \end{cases}$$

■

Анализ сложности рекурсивных алгоритмов

Введём рекурсивную функцию (на примере ханойских башенок)

$$T(n) = \begin{cases} t_0, n < N \text{ (нерекурсивная ветка)} \\ 2T(n-1) + 1 \end{cases} \Rightarrow T(n) = 2T(n-1) + 1$$

1. Метод подстановки

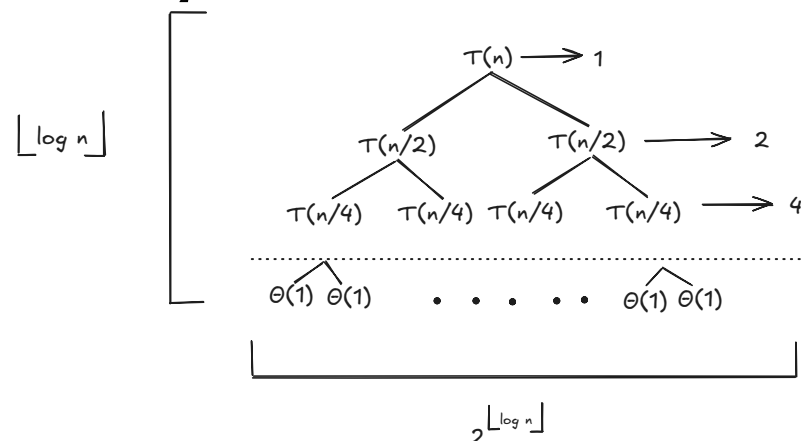
$$\begin{aligned} T(n) &= 2T(n-1) + 1 = 4T(n-2) + 2 + 1 \stackrel{i}{=} \\ &\stackrel{i}{=} i2^iT(n-i) + 2^{i-1} + 2^{i-2} + \dots + 1 = 2^n T(0) + \sum_{i=0}^{n-1} 2^i = \Theta(2^n) \end{aligned}$$

2. Метод математическое индукции

"Заметим, что сложность ..., докажем по математической индукции"

3. Рекурсивные деревья

$$T(n) = 2T\left(\frac{n}{2}\right) + 1$$



$$T(n) = \Theta(2^{\log n}) + \Theta\left(\sum_{i=0}^{\log n} 2^i\right) = \Theta(n) + \Theta(2^{\log n}) = \Theta(n) + \Theta(n) = \Theta(n)$$

№ 2.4.1

а) $x(n) = x(n-1) + 5$, для $n > 1$, $x(1) = 0$

$$\begin{aligned} x(n) &= x(n-1) + 5 = x(n-2) + 2 * 5 = x(n-3) + 3 * 5 \stackrel{i}{=} x(n-i) + 5i = \\ &= x(1) + 5n - 5 = 5(n-1) \end{aligned}$$

б) $x(n) = 3x(n-1)$, для $n > 1$, $x(1) = 4$

$$x(n) = 3 * x(n-1) = 3^2 * x(n-2) \stackrel{i}{=} 3^i * x(n-i) = 4 * 3^{n-1}$$

в) $x(n) = x(n-1) + n$, для $n > 0$, $x(0) = 0$

$$\begin{aligned} x(n) &= x(n-1) + n = x(n-2) + n + (n-1) \stackrel{i}{=} x(n-i) + n + (n-1) + \dots + (n-i) = \\ &= x(0) + n + (n-1) + \dots + 0 = \frac{n(n+1)}{2} \end{aligned}$$

$$\Gamma) x(n) = x\left(\frac{n}{2}\right) + n, \text{ для } n > 1, x(1) = 1, n = 2^k$$

$$x(n) = x\left(\frac{n}{2}\right) + n = x\left(\frac{x}{4}\right) + n + \frac{n}{2} = x\left(\frac{x}{2^i}\right) + n + \frac{n}{2} + \dots + \frac{n}{2^{i-1}} = 1 + n + \frac{n}{2} + \frac{n}{2^{k-1}} = \frac{2n - \frac{n}{2^k}}{2-1} = 2n -$$

$$\Delta) x(n) = x\left(\frac{n}{3}\right) + 1, \text{ для } n > 1, x(1) = 1, n = 3^k$$

$$x(n) = x\left(\frac{n}{3}\right) + 1 = \frac{n}{9} + 2 = \frac{n}{3^i} + i = x(1) + k = k + 1$$

Оценка сложности рекурсивных алгоритмов. Повторение

Общий вид рекурсивной функции

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Пример: Алгоритм Карацубы

$T(n)$ - время от n - количество бит

Операция	$T(n)$
+	$O(n)$
* (обычный)	$O(n^2)$
* (Карацубы)	$O(n^{\log_2 3})$

Алгоритм Карацубы

0. Считаю, что работаем в двоичной системе счисления, но можно обобщить

1. Считаю все числа одинаковой длины (дополняем ведущими 0)

2. Дополним 0 длину до степени 2

Пусть n - длина числа

$$A = A_1 * 2^{\frac{n}{2}} + A_2 = \overline{A_1 A_2}$$

$$\overline{a_n \dots a_0} * 2^{\frac{n}{2}} = \overline{a_n \dots a_0 0 \dots 0}, \text{ количество 0 в конце: } \frac{n}{2}$$

$$B = B_1 * 2^{\frac{n}{2}} + B_2 = \overline{B_1 B_2}$$

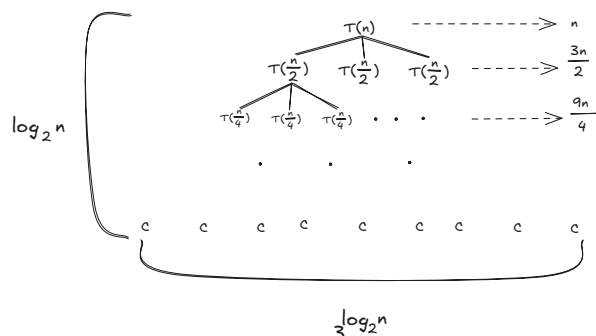
$$AB = (A_1 * 2^{\frac{n}{2}} + A_2)(B_1 * 2^{\frac{n}{2}} + B_2) = A_1 B_1 * 2^n + A_1 B_2 * 2^{\frac{n}{2}} + A_2 B_1 * 2^{\frac{n}{2}} + A_2 B_2 - \text{при}$$

такой записи 4 умножения

$$A_1 B_2 + A_2 B_1 = (A_1 + A_2)(B_1 + B_2) - A_1 B_1 - A_2 B_2 - \text{при такой записи 3 умножения}$$

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

Оценим сложность методом рекурсивных деревьев



$$T(n) = n^{\log_2 3} + \sum_{i=0}^{\log_2 n - 1} n \left(\frac{3}{2}\right)^i = n^{\log_2 3} + n \frac{\left(\frac{3}{2}\right)^{\log_2 n} - 1}{\frac{3}{2} - 1} =$$

$$= n^{\log_2 3} + 2n \left(\frac{3^{\log_2 n}}{n} - 1\right) = 3n^{\log_2 3} - 2n = \Theta(n^{\log_2 3})$$

Основной метод оценки. Мастер-теорема

Теорема. Пусть есть константы $a \geq 1$, $b > 1$, функция $f(n) > 0$

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Тогда:

1. Если $f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0 - const \Rightarrow T(n) = \Theta(n^{\log_b a})$
2. Если $f(n) = \Theta(n^{\log_b a}) \Rightarrow T(n) = \Theta(n^{\log_b a} \log n)$
3. Если

$$\begin{cases} f(n) = \Omega(n^{\log_b a + \varepsilon}), \varepsilon > 0 - const \\ \exists N \in \mathbb{N} : \forall n \geq N : af\left(\frac{n}{b}\right) \leq cf(n), c < 1 \text{ (условие регулярности)} \end{cases} \Rightarrow T(n) = \Theta(f(n))$$

Пример:

1. $T(n) = 8T\left(\frac{n}{2}\right) + 1000n^2$
 $1000n^2 = O(n^{3-0.5})$
 $\Rightarrow T(n) = \Theta(n^3)$
2. $T(n) = 2T\left(\frac{n}{2}\right) + 10n$
 $10n = \Theta(n^{\log_2 2})$
 $\Rightarrow T(n) = \Theta(n \log n)$
3. $T(n) = 2T\left(\frac{n}{2}\right) + n^2$
 $n^2 = \Omega(n^{1+0.7})$

Условие регулярности:

$$2f\left(\frac{n}{2}\right) \leq cf(n)$$

$$2 \frac{n^2}{4} \leq cn^2$$

$$c \geq \frac{1}{2}$$

$$N = 100, c = \frac{2}{3}$$

$$\Rightarrow T(n) = \Theta(n^2)$$

4. $T(n) = 2T\left(\frac{n}{2}\right) + n \log n$

Не подходит ни под какой пункт

Модификация 2 пункта:

$$f(n) = \Theta(n^{\log_b a} \log^k n) \Rightarrow T(n) = \Theta(n^{\log_b a} \log^{k+1} n), k \geq 0$$

$$\Rightarrow T(n) = \Theta(n \log^2 n)$$

$$5. T(n) = T\left(\frac{n}{2}\right) + n(2 - \cos n)$$

$$n(2 - \cos n) = \Omega(n^{0+0.5})$$

Условие регулярности:

$$1 \frac{n}{2}(2 - \cos \frac{n}{2}) \leq cn(2 - \cos n)$$

$$\frac{1}{2}(2 - \cos \frac{n}{2}) \leq c(2 - \cos n)$$

Пусть $n = 2\pi(2k + 1)$, $k \in \mathbb{Z}$, тогда

$$\frac{1}{2}(2 - \cos \frac{2\pi(2k+1)}{2}) \leq c(2 - \cos 2\pi(2k + 1))$$

$$\frac{3}{2} \leq c$$

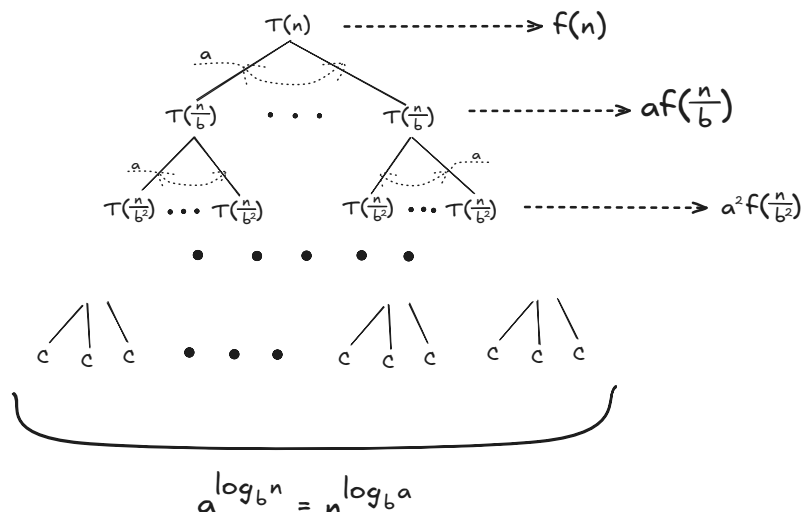
Но $c < 1$, противоречие $\Rightarrow$ не подходит

Доказательство мастер-теоремы

Special thanks to GNOM



1. Построим рекурсивное дерево исходной функции



$$\text{Тогда } T(n) = \Theta(n^{\log_b a}) + \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$$

$$\text{Пусть } S = \sum_{i=0}^{\log_b(n)-1} a^i f\left(\frac{n}{b^i}\right)$$

2. Введём ряд вспомогательных утверждений

$$1. \text{ Если } f(n) = O(n^{\log_b a - \varepsilon}) \Rightarrow S = O(n^{\log_b a})$$

$$2. \text{ Если } f(n) = \Theta(n^{\log_b a} \log^k n) \Rightarrow S = \Theta(n^{\log_b a} \log^{k+1} n)$$

$$3. \text{ Если } f(n) \text{ - регулярна } \Rightarrow S = \Theta(f(n))$$

3. Докажем введённые утверждения

$$1. S \leq \sum_{i=0}^{\log_b n - 1} a^i c \left(\frac{n}{b^i}\right)^{\log_b a - \varepsilon} = c \sum_{i=0}^{\log_b n - 1} \frac{a^i n^{\log_b a} n^{-\varepsilon}}{(b^i)^{\log_b a} (b^i)^{-\varepsilon}} = c \frac{n^{\log_b a}}{n^\varepsilon} \sum_{i=0}^{\log_b n - 1} \frac{a^i b^{i\varepsilon}}{a^i}$$

$$S \leq c \frac{n^{\log_b a}}{n^\varepsilon} \sum_{i=0}^{\log_b n - 1} (b^\varepsilon)^i = c \frac{n^{\log_b a}}{n^\varepsilon} \frac{b^{\varepsilon \log_b n} - 1}{b^\varepsilon - 1} \leq \frac{c}{b^\varepsilon - 1} \frac{n^{\log_b a}}{n^\varepsilon} n^\varepsilon =$$

$$= \dot{c} n^{\log_b a}, \quad \dot{c} = \frac{c}{b^\varepsilon - 1} \text{ (const)}$$

$$\Rightarrow S = O(n^{\log_b a})$$

$$2. S \sim \sum_{i=0}^{\log_b n - 1} a^i \left(\frac{n}{b^i}\right)^{\log_b a} \log^k \left(\frac{n}{b^i}\right) = n^{\log_b a} \sum_{i=0}^{\log_b n - 1} \frac{a^i}{b^{i \log_b a}} (\log^k n - \log^k b^i) = n^{\log_b a} \frac{1}{\log_b^k S} \sum_{i=0}^{\log_b n - 1}$$

$$\sum_{i=0}^{\log_b n - 1} (\log_b n - i)^k = \sum_{i=1}^{\log_b n} i^k$$

$$\text{Пусть } m = \log_b n, \quad \sum_{i=\frac{m}{2}}^m \left(\frac{m}{2}\right)^k \leq \sum_{i=\frac{m}{2}}^m i^k \leq \sum_{i=1}^m i^k \leq \sum_{i=1}^m m^k$$

$$\sum_{i=\frac{m}{2}}^m \left(\frac{m}{2}\right)^k = \left(\frac{m}{2}\right)^{k+1} = \frac{1}{2^{k+1}} m^{k+1}, \quad \sum_{i=1}^m m^k = m^{k+1} \Rightarrow \sum_{i=1}^m i^k = \Theta(m^{k+1})$$

$$\Rightarrow \sum_{i=0}^{\log_b n - 1} (\log_b n - i)^k = \Theta(\log_b^{k+1} n)$$

$$\Rightarrow S \sim C n^{\log_b a} \Theta(\log_b^{k+1} n), \quad C = \frac{1}{\log_b^k S}$$

$$\Rightarrow S = \Theta(n^{\log_b a} \log_b^{k+1} n)$$

3. $f(n)$ - регулярна, тогда

$$a^i f\left(\frac{n}{b^i}\right) = a^{i-1} \left(a f\left(\frac{n}{b^{i-1}}\right)\right) \leq a^{i-1} c f\left(\frac{n}{b^{i-1}}\right) \leq a^{i-2} c^2 f\left(\frac{n}{b^{i-2}}\right) \leq \dots \leq c^i a^{i-1} f\left(\frac{n}{b^{i-1}}\right)$$

$$a^i f\left(\frac{n}{b^i}\right) \leq c^i f(n)$$

$$\sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right) \leq \sum_{i=0}^{\log_b n - 1} c^i f(n) = f(n) \sum_{i=0}^{\log_b n - 1} c^i \leq f(n) \sum_{i=0}^{\infty} c^i = \frac{1}{1-c} f(n)$$

$$\Rightarrow S = \Theta(f(n))$$

4. Тогда для мастер-теоремы:

1. $T(n) = \Theta(n^{\log_b a}) + O(n^{\log_b a}) = \Theta(n^{\log_b a})$
2. $T(n) = \Theta(n^{\log_b a}) + \Theta(n^{\log_b a} \log^{k+1} n) = \Theta(n^{\log_b a} \log^{k+1} n)$
3. $T(n) = \Theta(n^{\log_b a}) + \Theta(f(n)) = \Theta(f(n)), f(n) = \Omega(n^{\log_b a + \varepsilon})$



Амортизационный анализ

Амортизационный анализ — оценка среднего времени операции, которая учитывает, что в последовательности действий редкие и «дорогие» операции компенсируются множеством «дешёвых».

Мы анализируем **не одну операцию, а последовательность операций**:

- суммарное время всех операций делим на их количество
- получаем **амортизированную стоимость одной операции**

$$a = \frac{\sum_{i=1}^n t_i}{n}, t_i - \text{время выполнения операции } i$$

Методы амортизационного анализа

Метод усреднения

Амортизационная стоимость определяется просто по исходной формуле.

Пример. Пусть дан массив длины N , для которого поддерживаются две операции:

1. $update(i, val)$ - изменить элемент в точке ($O(1)$)
2. $copy()$ - полностью скопировать текущий массив ($O(n)$)

Также разрешается делать операцию $copy$ не чаще, чем один раз каждые $K = \frac{N}{\log N}$.

Необходимо оценить амортизированную сложность одной операции.

Решение.

Пусть сделано M операций: R - количество вызовов $copy$, $M - R$ - количество вызовов $update$

Из-за ограничения на вызов *copy*: $(M - R) \geq K(R - 1)$, $R - 1$ - количество промежутков между вызовами *copy*

Считаем, что R - большое, тогда $R - 1 \approx R$

$$R(K + 1) \leq M$$

$$R \leq \frac{M}{K+1} \leq \frac{M}{K} = \frac{M \log N}{N}$$

$$\text{Сумма операций: } T(N) = (M - R) * 1 + R * N \leq M + M \log N$$

$$\text{Подставим в формулу: } a = \frac{T(n)}{M} = \frac{M + M \log N}{M} = 1 + \log N = O(\log N)$$

Метод предоплаты (метод бухгалтерского учёта)

Идея. Давайте каждой операции назначим амортизированную стоимость (в монетках), при этом эта стоимость для простых операций выше их реальной стоимости: избыток идёт в общую "копилку", а для тяжелых - наоборот: ниже реальной, они добирают монетки из "копилки". Тогда, если количество монет в "копилке" никогда не становится отрицательным, то амортизационная стоимость - верхняя оценка реальной средней стоимости.

Замечание. Для разных операций можно назначать разные амортизированные стоимости, тогда верхняя оценка одной (любого типа) операции - максимум из всех амортизированных стоимостей, а для конкретного типа - просто амортизированная стоимость.

Пример. Найти амортизированную стоимость одного *push_back* в *vector*

Решение.

push_back делится на два типа:

1. Легкий - просто добавили элемент ($O(1)$)
 2. Тяжелый - выделили новый массив размером $2C$, скопировали туда C старых элементов и добавили один новый ($O(C)$, C - размер *vector*)
- Давайте каждая операция *push_back* стоит 3 монеты (амортизированная сложность)

Приведем реальные стоимости:

1. Легкий - 1 монета
2. Тяжелый - $C + 1$ монета (+1, так как добавление одного элемента в конце - легкий *push_back*)

Тогда легкая операция приносит в копилку 2 монеты, а тяжелая забирает $C - 2$ монеты

Докажем, что копилка не будет пуста. Рассмотрим *push_back* с расширением массива длины C до длины $2C$:

До этого мы сделали C легких операций, значит в копилке - $2C$ монет.

$$2C - (C + 1) = C - 1 \geq 0$$

Несложно показать, что при увеличении C ($2C, 4C, \dots$) это также выполняется.

Следовательно $a = 3 = O(1)$

Метод потенциалов

φ_i - функция потенциала

Свойства:

1. $\varphi_0 = 0$
2. $\varphi_i \geq 0$

Пусть t_i - реальное время

Пересчитаем амортизированное время по формуле:

$$t_i^* = t_i + \Delta\varphi_i = t_i + (\varphi_i - \varphi_{i-1})$$

$$\sum_{i=1}^n t_i^* = \sum_{i=1}^n t_i + \sum_{i=1}^n (\varphi_i - \varphi_{i-1}) = \sum_{i=1}^n t_i + \varphi_n - \varphi_0 \geq \sum_{i=1}^n t_i$$

Пример. *push_back* в *vector*

$$t_i = \begin{cases} 1, i \neq 2^k + 1 \\ i, i = 2^k + 1 \end{cases}$$

$$t_i = \begin{cases} 1, size_{i-1} < cap_{i-1} & (1) \\ cap_{i-1} + 1, size_{i-1} = cap_{i-1} & (2) \end{cases}$$

$$\varphi_i = 2size_i - cap_i$$

$$(1) t_i^* = 1 + 2 = 3$$

$$(2) t_i^* = cap_{i-1} + 1 + 2(size_i - size_{i-1}) - (cap_i - cap_{i-1}) = cap_{i-1} + 1 + 2 - (2cap_{i-1} - cap_{i-1}) = 3$$

(1) - медленное увеличение потенциала

(2) - резкое падение потенциала

Пример. Есть очередь на двух стеках. Доказать асимптотику *push/pop* методом потенциалов

pop:

$$t_i = \begin{cases} 1, s_2^{i-1} > 0 & (1) \\ s_1^{i-1} + 1, s_2^{i-1} = 0 & (2) \end{cases}$$

<i>pop</i>	s_1^{i-1}	s_2^{i-1}	s_1^i	s_2^i
(1)				$s_2^{i-1} - 1$
(2)		0	0	$s_1^{i-1} - 1$

$$\varphi_i = s_1^i$$

pop:

$$(1) t_i^* = 1 + s_1^i - s_1^{i-1} = 1 + s_1^i - s_1^i = 1$$

$$(2) t_i^* = s_1^{i-1} + 1 + 0 - s_1^{i-1} = 1$$

push:

$$t_i = 1$$

$$\varphi_i = s_1^i$$

$$t_i^* = 1 + s_1^i - s_1^{i-1} = 1 + s_1^{i-1} + 1 - s_1^{i-1} = 2$$

Пример. Splay-дерево

Теорема. $T(\text{splay}) = O(\log n)$ амортизировано



Ранг вершины: $r(x) = \log_2 c(x)$, $c(x)$ - количество вершин в поддереве x .

$$\varphi = \sum_x r(x)$$

Утверждения:

1. $T(\text{zig}) \leq 1 + 3r'(x) - 3r(x)$
2. $T(\text{zig} - \text{zig}) \leq 3r'(x) - 3r(x)$
3. $T(\text{zig} - \text{zag}) \leq 3r'(x) - 3r(x)$

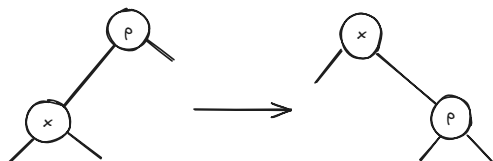
$r'(x)$ - ранг вершины x после поворота

Докажем, пользуясь утверждениями:

$$\begin{aligned} \sum t_i &= T(\text{zig}) + \sum T(\text{zig} - \text{zig}) \leq 1 + 3r_n(x) - 3r_{n-1}(x) + \sum_{i=1}^{n-1} (3r_i(x) - 3r_{i-1}(x)) = \\ &= 1 + 3r_n(x) - 3r_{n-1}(x) + 3r_{n-1}(x) - 3r_0(x) = 1 + 3r_n(x) - 3r_0(x) \leq 1 + 3r_n(x) = \\ &= 1 + 3 \log C(x) = 1 + \log N, \quad N - \text{ всего вершин в дереве} \end{aligned}$$

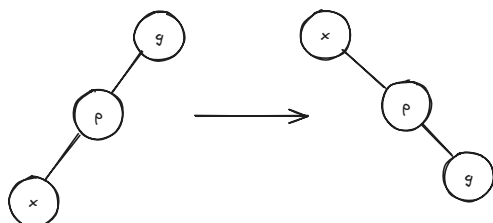
Докажем утверждения:

1. *zig*



$$t^* = 1 + r'(x) + r'(p) - r(x) - r(p) = 1 + r'(p) - r(x) \leq 1 + r'(x) - r(x) \leq 1 + 3r'(x) - 3r(x)$$

2. *zig - zig*



$$\begin{aligned} t^* &= 2 + r'(x) + r'(p) + r'(g) - r(x) - r(p) - r(g) = 2 + r'(p) + r'(g) - r(x) - r(p) \\ 2 + r'(p) + r'(g) - r(x) - r(p) &\leq 2 + r'(p) + r'(g) - 2r(x) \leq 2 + r'(x) + r'(g) - 2r(x) \end{aligned}$$

Нужно доказать:

$$2 + r'(x) + r'(g) - 2r(x) \leq 3(r'(x) - r(x))$$

$$r'(g) + r(x) - 2r'(x) \leq -2$$

$$\log c'(g) + \log c(x) - 2 \log c'(x) \leq -2$$

$$\log \frac{c'(g)}{c'(x)} + \log \frac{c(x)}{c'(x)} \leq -2$$

Пусть $\alpha = \frac{c'(g)}{c'(x)}$, $\beta = \frac{c(x)}{c'(x)}$

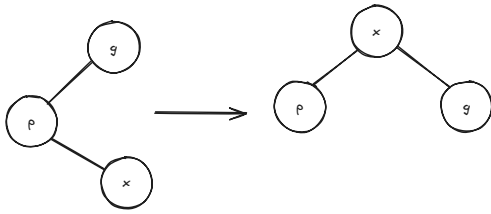
$$\alpha + \beta \leq 1$$

$$\sqrt{\alpha\beta} \leq \frac{\alpha+\beta}{2}$$

$$\alpha\beta \leq \frac{(\alpha+\beta)^2}{4} \leq \frac{1}{4}$$

$$\text{Тогда } \log \alpha\beta \leq \log \frac{1}{4} = -2$$

3. zig – zag



$$t^* = 2 + r'(x) + r'(p) + r'(g) - r(x) - r(p) - r(g) \leq 2 + r'(p) + r'(g) - 2r(x)$$

$$2 + r'(p) + r'(g) - 2r(x) \leq 2r'(x) - 2r(x)$$

$$r'(p) + r'(g) - 2r'(x) \leq -2$$

$$\log \frac{c'(p)}{c'(x)} + \log \frac{c'(g)}{c'(x)} \leq -2$$

$$\text{Пусть } \alpha = \frac{c'(p)}{c'(x)}, \quad \beta = \frac{c'(g)}{c'(x)}$$

$$\alpha + \beta \leq 1$$

$$\sqrt{\alpha\beta} \leq \frac{\alpha+\beta}{2}$$

$$\alpha\beta \leq \frac{(\alpha+\beta)^2}{4} \leq \frac{1}{4}$$

$$\log \alpha\beta \leq \log \frac{1}{4} = -2$$

■